

Código limpo

Exercício - Java Pro / Módulo 4

1. Nomenclatura adequada

O nome deve ajudar a entender a intenção sem abrir toda a implementação. Uma variável chamada `quantidadeTens` informa mais que `x`. Isso reduz a chance de usar um valor de forma errada e facilita a revisão. O nome precisa combinar com o domínio e com o comportamento real; um método chamado `consultar` não deveria alterar dados silenciosamente.

2. Causa raiz

Corrigir só o sintoma pode esconder o defeito e fazer o problema voltar em outro ponto. Se uma venda fica com total negativo, limitar o resultado a zero não explica o erro. É preciso investigar, por exemplo, se uma quantidade negativa entrou no sistema. Depois da correção, um teste deve reproduzir o caso para evitar a regressão.

3. Política do escoteiro

A ideia é deixar o código um pouco melhor do que estava. Ao mexer em uma função, posso melhorar um nome ou retirar uma duplicação pequena que atrapalhe a leitura. Isso deve ser feito com escopo controlado e testes, sem transformar toda alteração simples em uma reescrita do sistema.

4. `somaNumeros(int a, int b, int c, int d, int e, int f)`

A assinatura `private void somaNumeros(...)` recebe muitos parâmetros posicionais. Fica fácil trocar a ordem ou esquecer um valor. Para uma soma de quantidade variável de inteiros, usaria `int somar(int... numeros)`, com uma regra explícita para overflow. O retorno `void` também merece revisão: se a função apenas calcula a soma, faz mais sentido devolver o resultado e deixar a impressão para quem chamou. Se ela tiver outro efeito, isso precisa ficar claro no nome.

5. `oPaiTaOn()`

O nome usa uma expressão informal que não explica o propósito do método. A pessoa que lê precisa procurar a implementação para descobrir o que ele faz. O nome correto depende desse comportamento. Se a intenção for verificar se um usuário está conectado, por exemplo, `boolean estaConectado()` comunica melhor a consulta; não dá para afirmar que esse é o comportamento apenas pela assinatura original.

6. `checaSaldoEAtualiza(long userId, double value)`

O nome já mistura consultar e alterar. Isso fere a separação entre consulta e comando e dificulta prever os efeitos da chamada. Separaria a consulta de saldo da operação de débito ou crédito. A operação de alteração, porém, deve validar o saldo e gravar a mudança na mesma transação; consultar antes e atualizar depois, sem proteção, permite uma corrida entre requisições. Também evitaria `double` para dinheiro: usaria `BigDecimal` ou um valor inteiro em centavos, com regras de arredondamento definidas.